

tf.compat.v1.lookup.StaticHashTable

Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/compat/v1/lookup/StaticHashTable

A generic hash table that is immutable once initialized.

Function Signatures

```
tf.compat.v1.lookup.StaticHashTable( initializer,  
default_value, name=None, experimental_is_anonymous=False )  
tf.tables_initializer()  
lookup()
```

Code Examples

```
tf.compat.v1.lookup.StaticHashTable(  
initializer, default_value, name=None,  
experimental_is_anonymous=False  
)
```

```
tf.compat.v1.lookup.StaticHashTable(  
initializer, default_value, name=None,  
experimental_is_anonymous=False  
)
```

```
keys_tensor = tf.constant([1, 2])
vals_tensor = tf.constant([3, 4])
input_tensor = tf.constant([1, 5])
table = tf.lookup.StaticHashTable(
    tf.lookup.KeyValueTensorInitializer(keys_tensor, vals_tensor), -1)
out = table.lookup(input_tensor)
with tf.Session() as sess:
    sess.run(tf.tables_initializer())
    print(sess.run(out))
```

```
keys_tensor = tf.constant([1, 2])
vals_tensor = tf.constant([3, 4])
input_tensor = tf.constant([1, 5])
table = tf.lookup.StaticHashTable(
    tf.lookup.KeyValueTensorInitializer(keys_tensor, vals_tensor), -1)
out = table.lookup(input_tensor)
with tf.Session() as sess:
    sess.run(tf.tables_initializer())
    print(sess.run(out))
```

```
keys_tensor = tf.constant([1, 2])
vals_tensor = tf.constant([3, 4])
input_tensor = tf.constant([1, 5])
table = tf.lookup.StaticHashTable(
    tf.lookup.KeyValueTensorInitializer(keys_tensor, vals_tensor), -1,
    experimental_is_anonymous=True)
with tf.control_dependencies([tf.tables_initializer()]):
    out = table.lookup(input_tensor)
with tf.Session() as sess:
    print(sess.run(out))
```

```
keys_tensor = tf.constant([1, 2])
vals_tensor = tf.constant([3, 4])
input_tensor = tf.constant([1, 5])
table = tf.lookup.StaticHashTable(
    tf.lookup.KeyValueTensorInitializer(keys_tensor, vals_tensor), -1,
    experimental_is_anonymous=True)
with tf.control_dependencies([tf.tables_initializer()]):
    out = table.lookup(input_tensor)
with tf.Session() as sess:
    print(sess.run(out))
```

```
tf.enable_eager_execution()
keys_tensor = tf.constant([1, 2])
vals_tensor = tf.constant([3, 4])
input_tensor = tf.constant([1, 5])
table = tf.lookup.StaticHashTable(
    tf.lookup.KeyValueTensorInitializer(keys_tensor, vals_tensor), -1)
print(table.lookup(input_tensor))
```

```
tf.enable_eager_execution()
keys_tensor = tf.constant([1, 2])
vals_tensor = tf.constant([3, 4])
input_tensor = tf.constant([1, 5])
table = tf.lookup.StaticHashTable(
    tf.lookup.KeyValueTensorInitializer(keys_tensor, vals_tensor), -1)
print(table.lookup(input_tensor))
```

Documentation

A generic hash table that is immutable once initialized.

Inherits From: StaticHashTable, TrackableResource

When running in graph mode, you must evaluate the tensor returned by `tf.tables_initializer()` before evaluating the tensor returned by this class's `lookup()` method. Example usage in graph mode:

Note that in graph mode if you set `experimental_is_anonymous` to `True`, you should only call `Session.run` once, otherwise each `Session.run` will create (and destroy) a new table unrelated to each other, leading to errors such as "Table not initialized". You can do so like this:

In eager mode, no special code is needed to initialize the table. Example usage in eager mode:

Attributes

`## Methods`

export

[View source](#)

Returns tensors of all keys and values in the table.

lookup

[View source](#)

Looks up keys in a table, outputs the corresponding values.

The `default_value` is used for keys not present in the table.

[View source](#)

Compute the number of elements in this table.

__getitem__

[View source](#)

Looks up keys in a table, outputs the corresponding values.